LAPORAN PRAKTIKUM

STRUKTUR DATA

SINGLE LINKED LIST

PEKAN 5

Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis sampaikan kehadirat Allah SWT. Salawat dan salam disampaikan kepada Nabi Muhammad SAW. Karena thaufik dan hidayah-Nya, laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban dari kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar serta penerapan bahasa pemrograman Java.

Melalui praktikum ini, penulis memperoleh pengalaman langsung dalam memahami Implementasi Single Linked List menggunakan Java. Diharapkan laporan ini dapat memberikan gambaran yang jelas mengenai materi yang telah dipelajari serta hasil dari percobaan yang dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajiannya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta semua pihak yang telah memberikan bimbingan, arahan, dan dukungan sehingga laporan praktikum ini dapat tersusun. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami dasar-dasar pemrograman Java.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....i

DAFTAR ISI.....ii

DAFTAR LAMPIRAN.....iii

BAB 1 PENDAHULUAN

1.1 Latar Belakang..... 1

1.2 Tujuan Praktikum.....2

1.3 Manfaat Praktikum.....2

BAB 2 PEMBAHASAN

2.1 Dasar Teori.....3

2.2 Kode Program..... 5

2.3 Langkah Kerja.....10

2.4 Analisis Hasil..... 22

BAB 3 KESIMPULAN

3.1 Hasil Praktikum.....23

3.2 Saran Pengembangan..... 26

DAFTAR KEPUSTAKAAN.....27

TUGAS PRAKTIKUM PEKAN 5.....28

DAFTAR LAMPIRAN

Kode Program 2.1 (Kode Program NodeSLL)..... 5

Kode Program 2.2 (Kode Program TambahSLL)..... 6

Kode Program 2.3 (Kode Program HapusSLL).....8

Kode Program 2.4 (Kode Program PencarianSLL)..... 8

Gambar 2.1 (Langkah 1 NodeSLL)..... 10

Gambar 2.2 (Langkah 2 NodeSLL)..... 10

Gambar 2.3 (Langkah 3 NodeSLL)..... 11

Gambar 2.4 (Langkah 1 TambahSLL)..... 11

Gambar 2.5 (Langkah 2 TambahSLL)..... 12

Gambar 2.6 (Langkah 3 TambahSLL)..... 12

Gambar 2.7 (Langkah 4 TambahSLL)..... 12

Gambar 2.8 (Langkah 5 TambahSLL)..... 13

Gambar 2.9 (Langkah 6 TambahSLL)..... 13

Gambar 2.10 (Langkah 7 TambahSLL)..... 13

Gambar 2.11 (Langkah 8 TambahSLL)..... 14

Gambar 2.12 (Langkah 9 TambahSLL)..... 14

Gambar 2.13 (Langkah 10 TambahSLL)..... 14

Gambar 2.14 (Langkah 11 TambahSLL)..... 15

Gambar 2.15 (Langkah 1 HapusSLL).....15

Gambar 2.16 (Langkah 2 HapusSLL).....16

Gambar 2.17 (Langkah 3 HapusSLL).....16

Gambar 2.18 (Langkah 4 HapusSLL).....16

Gambar 2.19 (Langkah 5 HapusSLL).....17

Gambar 2.20 (Langkah 6 HapusSLL).....17
Gambar 2.21 (Langkah 7 HapusSLL).....17
Gambar 2.22 (Langkah 8 HapusSLL).....18
Gambar 2.23 (Langkah 9 HapusSLL).....18
Gambar 2.24 (Langkah 10 HapusSLL).....18
Gambar 2.25 (Langkah 11 HapusSLL).....19
Gambar 2.26 (Langkah 1 PencarianSLL)..... 19
Gambar 2.27 (Langkah 2 PencarianSLL)..... 20
Gambar 2.28 (Langkah 3 PencarianSLL)..... 20
Gambar 2.29 (Langkah 4 PencarianSLL)..... 21
Gambar 2.30 (Langkah 5 PencarianSLL)..... 21
Gambar 2.31 (Langkah 6 PencarianSLL)..... 21
Gambar 3.1 (Hasil Praktikum TambahSLL).....23
Gambar 3.2 (Hasil Praktikum HapusSLL).....24
Gambar 3.3 (Hasil Praktikum PencarianSLL).....25

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengelolaan data menjadi salah satu hal yang penting, terutama ketika jumlah data terus bertambah dan perlu diakses secara dinamis. Salah satu struktur data yang sering digunakan untuk kebutuhan tersebut adalah Single Linked List. Berbeda dengan array yang memiliki ukuran tetap, linked list memberikan fleksibilitas karena elemen dapat ditambahkan atau dihapus tanpa harus menggeser data lain secara langsung. Hal ini membuat linked list lebih efisien dalam kondisi tertentu, terutama ketika sering terjadi perubahan data.

Single Linked List sendiri terdiri dari sekumpulan node yang saling terhubung melalui pointer, di mana setiap node menyimpan data dan alamat ke node berikutnya. Konsep ini terlihat sederhana, tetapi dalam implementasinya membutuhkan pemahaman yang cukup terkait pengelolaan pointer dan alur data. Kesalahan kecil, seperti pointer yang tidak terhubung dengan benar, dapat menyebabkan data tidak dapat diakses atau bahkan hilang. Oleh karena itu, praktikum ini menjadi penting untuk memahami bagaimana struktur ini bekerja secara langsung melalui kode program.

Melalui praktikum ini, mahasiswa tidak hanya mempelajari konsep dasar Single Linked List secara teori, tetapi juga melihat bagaimana operasi-operasi seperti penambahan, penghapusan, penelusuran, dan pencarian data dilakukan. Dengan mencoba sendiri implementasinya, pemahaman terhadap alur kerja linked list menjadi lebih jelas dan tidak hanya bersifat abstrak. Hal ini diharapkan dapat membantu dalam memahami struktur data lain yang memiliki konsep serupa.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami konsep dasar Single Linked List beserta struktur penyusunnya.
2. Mengimplementasikan operasi dasar seperti penambahan, penghapusan, penelusuran, dan pencarian data pada linked list.
3. Melatih kemampuan dalam mengelola pointer dan alur data pada struktur data dinamis.

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Membantu mahasiswa memahami cara kerja struktur data dinamis secara lebih nyata melalui implementasi langsung.
2. Meningkatkan kemampuan logika dalam mengelola data, terutama dalam proses manipulasi node pada linked list.
3. Menjadi dasar pemahaman untuk mempelajari struktur data lain yang lebih kompleks.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Struktur Data Single Linked List merupakan salah satu bentuk struktur data dinamis yang digunakan untuk menyimpan sekumpulan data secara berurutan. Berbeda dengan array yang memiliki ukuran tetap dan penyimpanan berurutan di memori, linked list terdiri dari node-node yang saling terhubung melalui pointer. Setiap node menyimpan dua bagian utama, yaitu data dan alamat (referensi) ke node berikutnya. Konsep ini membuat linked list lebih fleksibel karena ukuran data dapat bertambah atau berkurang tanpa perlu mengalokasikan ulang memori secara keseluruhan.

Single Linked List bekerja dengan prinsip keterhubungan satu arah, di mana setiap node hanya memiliki satu pointer yang menunjuk ke node selanjutnya. Akses data pada struktur ini selalu dimulai dari node pertama yang disebut head, kemudian dilanjutkan ke node berikutnya hingga mencapai node terakhir yang memiliki pointer bernilai null. Karena tidak ada akses langsung ke elemen tertentu seperti pada array, proses pencarian atau penelusuran data dilakukan secara berurutan (linear). Hal ini membuat linked list lebih cocok digunakan pada kondisi yang membutuhkan fleksibilitas penambahan dan penghapusan data dibandingkan kecepatan akses.

Dalam implementasinya, terdapat beberapa operasi dasar pada Single Linked List, yaitu:

1. insert (penambahan), untuk menambahkan node baru baik di depan (insert at front), di belakang (insert at end), maupun di posisi tertentu,
2. delete (penghapusan), untuk menghapus node dari depan, belakang, atau posisi tertentu dalam list,
3. traversal (penelusuran), untuk menampilkan atau mengakses seluruh data dalam linked list secara berurutan,

4. search (pencarian), untuk mencari data tertentu dalam list dengan membandingkan setiap node.

Proses kerja pada beberapa operasi tersebut juga memiliki langkah-langkah tertentu. Pada operasi penambahan di depan, langkahnya adalah:

1. membuat node baru,
2. mengarahkan pointer node baru ke head lama,
3. memperbarui head menjadi node baru.

Sedangkan pada penambahan di akhir, langkahnya adalah:

1. membuat node baru,
2. menelusuri hingga node terakhir,
3. menghubungkan node terakhir dengan node baru.

Untuk operasi penghapusan, langkahnya adalah:

1. menentukan node yang akan dihapus,
2. mengubah pointer node sebelumnya agar melewati node yang dihapus,
3. menyesuaikan head jika node yang dihapus berada di awal.

Dalam implementasinya, Single Linked List banyak digunakan karena kemampuannya dalam mengelola data secara dinamis tanpa batasan ukuran tetap. Meskipun akses datanya lebih lambat dibandingkan array, kelebihan dalam fleksibilitas dan efisiensi manipulasi data menjadikan struktur ini cukup penting dalam berbagai kasus pemrograman.

2.2 Kode Program

2.2.1 NodeSLL_2511531016

```
package pekan5_2511531016;

public class NodeSLL_2511531016 {
// Node bagian data
int data_1016;
// Pointer ke node berikutnya
NodeSLL_2511531016 next_1016;
// Konstruktor menginisialisasi node dengan data
public NodeSLL_2511531016(int data_1016) {
this.data_1016 = data_1016;
this.next_1016 = null;
}
}
```

Kode Program 2.1 (Kode Program NodeSLL)

2.2.2 TambahSLL_2511531016

```
package pekan5_2511531016;

public class TambahSLL_2511531016 {
    public static NodeSLL_2511531016 insertAtFront_1016(NodeSLL_2511531016 head_1016, int value_1016) {
        NodeSLL_2511531016 new_node = new NodeSLL_2511531016(value_1016);
        new_node.next_1016 = head_1016;
        return new_node;
    }

    // Fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511531016 insertAtEnd_1016(NodeSLL_2511531016 head_1016, int value_1016) {
        // Buat sebuah node dengan sebuah nilai
        NodeSLL_2511531016 newNode = new NodeSLL_2511531016(value_1016);
        // Jika list kosong  maka node jadi head
        if (head_1016 == null) {
            return newNode;
        }
        // Simpan head ke variabel sementara
        NodeSLL_2511531016 last = head_1016;
        // Telusuri ke node akhir
        while (last.next_1016 != null) {
            last = last.next_1016;
        }
        // Ubah pointer
        last.next_1016 = newNode;
        return head_1016;
    }

    static NodeSLL_2511531016 GetNode_1016(int data_1016) {
        return new NodeSLL_2511531016(data_1016);
    }

    static NodeSLL_2511531016 insertPos_1016(NodeSLL_2511531016 headNode, int position_1016, int value_1016) {
        NodeSLL_2511531016 head_1016 = headNode;
        if (position_1016 < 1)
```

```java
            System.out.print ("Invalid position");
        if (position_1016 == 1) {
            NodeSLL_2511531016 new_node = new NodeSLL_2511531016(value_1016);
            new_node.next_1016 = head_1016;
            return new_node;
        } else {
            while (position_1016-- != 0) {
                if (position_1016 == 1) {
                    NodeSLL_2511531016 newNode = GetNode_1016 (value_1016);
                    newNode.next_1016 = headNode.next_1016;
                    headNode.next_1016 = newNode;
                    break;
                }
                headNode = headNode.next_1016;
                }
                if (position_1016 != 1)
                    System.out.print("Posisi di luar jangkauan");
                    return head_1016;
        }
    }
    public static void printList_1016 (NodeSLL_2511531016 head_1016) {
        NodeSLL_2511531016 curr_1016 = head_1016;
        while (curr_1016.next_1016 != null) {
            System.out.print(curr_1016.data_1016+"-->");
            curr_1016 = curr_1016.next_1016;
        }
        if (curr_1016.next_1016==null) {
        System.out.print(curr_1016.data_1016);  }
        System.out.println();
        }

    public static void main (String[] args) {
        // Buat linked list 2->3->5->6
        NodeSLL_2511531016 head_1016 = new NodeSLL_2511531016(2);
        head_1016.next_1016 = new NodeSLL_2511531016(3);
       head_1016.next_1016.next_1016 = new NodeSLL_2511531016(5);
       head_1016.next_1016.next_1016.next_1016 = new NodeSLL_2511531016(6);
       // Cetak list asli
       System.out.print("Senarai berantai awal:");
       printList_1016 (head_1016);
       // Tambahkan node baru didepan
       System.out.print("tambah 1 simpul di depan: ");
       int data_1016 = 1;
       head_1016 = insertAtFront_1016(head_1016, data_1016);
       // Cetak update list
       printList_1016(head_1016);
       // Tambahkan node baru dibelakang
       System.out.print("tambah 1 simpul di belakang");
       int data2 = 7;
       head_1016 = insertAtEnd_1016 (head_1016, data2);
       // Cetak update list
       printList_1016 (head_1016);
       System.out.print("tambah 1 simpul ke data 4: ");
       int data3 = 4;
       int pos = 4;
       head_1016 = insertPos_1016 (head_1016, pos, data3);
       // Cetak update list
       printList_1016 (head_1016);
    }
}
```

Kode Program 2.2 (Kode Program TambahSLL)

2.2.3 HapusSLL_2511531016

```java
package pekan5_2511531016;

public class HapusSLL_2511531016 {
    // Fungsi untuk menghapus head
    public static NodeSLL_2511531016 deleteHead_1016 (NodeSLL_2511531016 head_1016) {
        // Jika SLL kosong
        if (head_1016 == null)
            return null;
        // Pindahkan head ke node berikutnya
        head_1016 = head_1016.next_1016;
        // Return head baru
        return head_1016;
    }

    // Fungsi menghapus node terakhir SLL
    public static NodeSLL_2511531016 removeLastNode_1016 (NodeSLL_2511531016 head_1016) {
        // Jika list kosong, return null
        if (head_1016 == null) {
            return null;
        }
        // Jika list satu node, hapus node dan return null
        if (head_1016 == null) {
            return null;
        }
        // Temukan node terakhir ke dua
        NodeSLL_2511531016 secondLast = head_1016;
        while (secondLast.next_1016.next_1016 != null) {
            secondLast = secondLast.next_1016;
        }
        // Hapus node terakhir
        secondLast.next_1016 = null;
        return head_1016;
    }

     // Fungsi menghapus node di posisi tertentu
     public static NodeSLL_2511531016 deleteNode_1016 (NodeSLL_2511531016 head_1016, int position_1016) {
         NodeSLL_2511531016 temp_1016 = head_1016;
         NodeSLL_2511531016 prev_1016 = null;
         // Jika linked list null
         if (temp_1016 == null)
             return head_1016;
         // Kasus 1: head dihapus
         if (position_1016 == 1) {
             head_1016 = temp_1016.next_1016;
             return head_1016;
             }

         // Kasus 2: menghapus node di tengah
         // Telusuri ke node yang dihapus
         for (int i_1016 = 1; temp_1016 != null && i_1016 < position_1016; i_1016++) {
              prev_1016 = temp_1016;
              temp_1016 = temp_1016.next_1016;
         }

         // jika ditemukan, hapus node
         if (temp_1016 != null) {
              prev_1016.next_1016 = temp_1016.next_1016;
         } else {
                 System.out.println("Data tidak ada");
             }
         return head_1016;
         }

         // Fungsi mencetak SLL
         public static void printList_1016(NodeSLL_2511531016 head_1016) {
             NodeSLL_2511531016 curr_1016 = head_1016;
             while (curr_1016.next_1016 != null) {
                 System.out.print(curr_1016.data_1016 + "-->");
                 curr_1016 = curr_1016.next_1016;
```

```java
        }
        System.out.print(curr_1016.data_1016);
        System.out.println();
    }

    // Kelas main
    public static void main(String[] args) {
        // Buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
        NodeSLL_2511531016 head_1016 = new NodeSLL_2511531016(1);
        head_1016.next_1016 = new NodeSLL_2511531016(2);
        head_1016.next_1016.next_1016 = new NodeSLL_2511531016(3);
        head_1016.next_1016.next_1016.next_1016 = new NodeSLL_2511531016(4);
        head_1016.next_1016.next_1016.next_1016.next_1016 = new NodeSLL_2511531016(5);
        head_1016.next_1016.next_1016.next_1016.next_1016.next_1016 = new NodeSLL_2511531016(6);

        // Cetak list awal
        System.out.println("List awal: ");
        printList_1016(head_1016);

        // Hapus head
        head_1016 = deleteHead_1016(head_1016);
        System.out.println("List setelah head dihapus: ");
        printList_1016(head_1016);

        // Hapus node terakhir
        head_1016 = removeLastNode_1016(head_1016);
        System.out.println("List setelah simpul terakhir dihapus: ");
        printList_1016(head_1016);

        // Hapus node di posisi 2
        int position_1016 = 2;
        head_1016 = deleteNode_1016(head_1016, position_1016);
        System.out.println("List setelah posisi 2 dihapus: ");
        printList_1016(head_1016);
    }
```

Kode Program 2.3 (Kode Program HapusSLL)

2.2.4 PencarianSLL_2511531016

```java
package pekan5_2511531016;

public class PencarianSLL_2511531016 {
    static boolean searchKey_1016 (NodeSLL_2511531016 head_1016, int key_1016) {
        NodeSLL_2511531016 curr_1016 = head_1016;
        while (curr_1016 != null) {
            if (curr_1016.data_1016 == key_1016)
                return true;
            curr_1016 = curr_1016.next_1016;
            }
        return false;
        }

    public static void traversal_1016 (NodeSLL_2511531016 head_1016) {
        // Mulai dari head
        NodeSLL_2511531016 curr_1016 = head_1016;
        // Telusuri sampai pointer null
        while (curr_1016 != null)  {
            System.out.print(" " + curr_1016.data_1016);
            curr_1016 = curr_1016.next_1016;
            }
        System.out.println();
        }

    public static void main (String[] args) {
        NodeSLL_2511531016 head_1016 = new NodeSLL_2511531016(14);
        head_1016.next_1016 = new NodeSLL_2511531016(21);
        head_1016.next_1016.next_1016 = new NodeSLL_2511531016(13);
        head_1016.next_1016.next_1016.next_1016 = new NodeSLL_2511531016(30);
        head_1016.next_1016.next_1016.next_1016.next_1016 = new NodeSLL_2511531016(10);
        System.out.print ("Penelusuran SLL : ") ;
        traversal_1016 (head_1016);
        // Data yang akan dicari
        int key_1016 = 30;
        System.out.print("cari data " + key_1016 + "  =  ");
```

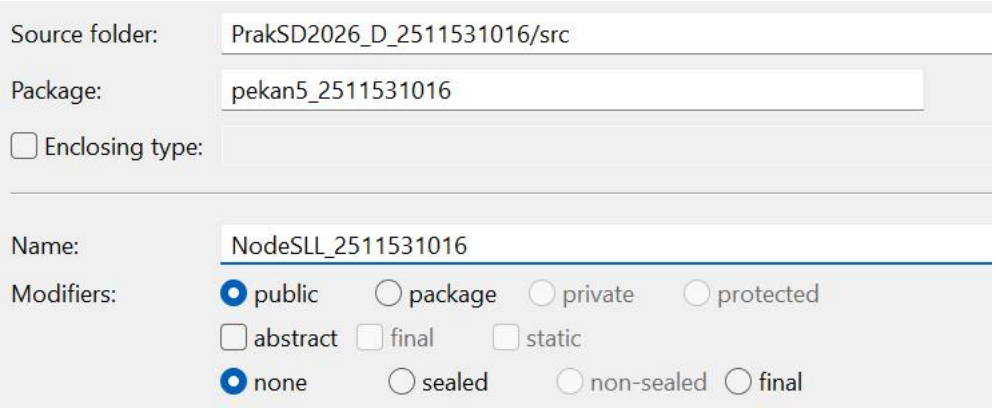
```
        System.out.print ("Penelusuran SLL : ") ;
        traversal_1016 (head_1016);
        // Data yang akan dicari
        int key_1016 = 30;
        System.out.print("cari data " + key_1016 + "  =  ");
        if (searchKey_1016(head_1016, key_1016))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}
```

Kode Program 2.4 (Kode Program PencarianSLL)

2.3 Langkah Kerja

2.3.1 NodeSLL_2511531016

1. Membuat kelas baru bernama *NodeSLL_2511531016* dalam *package pekan5_2511531016.*



Gambar 2.1 (Langkah 1 NodeSLL)

2. Program diawali dengan deklarasi package dan pembuatan class. Lalu, program mendeklarasikan atribut pada node yang terdiri dari data dan pointer ke node berikutnya, di mana data_1016 digunakan untuk menyimpan nilai, sedangkan next_1016 berfungsi sebagai penunjuk ke node berikutnya dalam struktur Single Linked List.

```
package pekan5_2511531016;

public class NodeSLL_2511531016 {
// Node bagian data
int data_1016;
// Pointer ke node berikutnya
NodeSLL_2511531016 next_1016;
```

Gambar 2.2 (Langkah 2 NodeSLL)

3. Kemudian, Konstruktor digunakan untuk menginisialisasi nilai node saat pertama kali dibuat, yaitu dengan mengisi data ke dalam node melalui

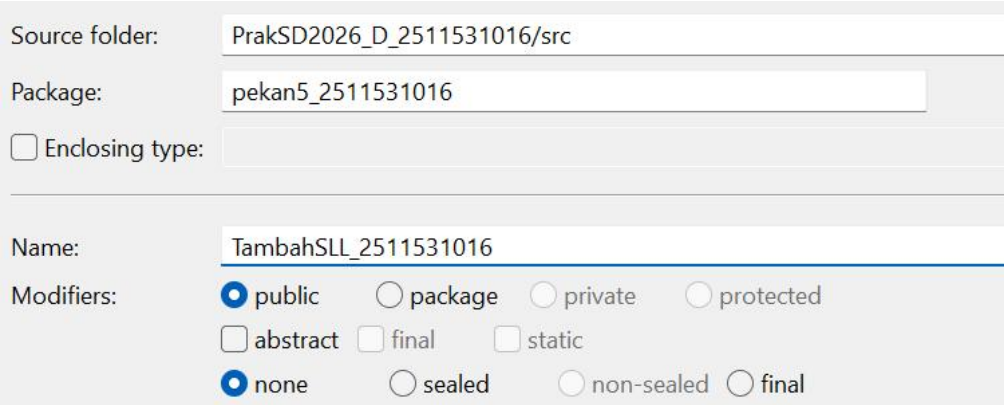
this.data_1016 = data_1016; dan mengatur this.next_1016 = null; yang menandakan node belum terhubung ke node lain.

```
// Konstruktor menginisialisasi node dengan data
public NodeSLL_2511531016(int data_1016) {
this.data_1016 = data_1016;
this.next_1016 = null;
}
```

Gambar 2.3 (Langkah 3 NodeSLL)

2.3.2 TambahSLL_2511531016

1. Membuat kelas baru bernama *TambahSLL_2511531016* dalam *package pekan5_2511531016.*



Gambar 2.4 (Langkah 1 TambahSLL)

2. Deklarasikan package dan class. Lalu, buat method untuk menambahkan node di bagian depan linked list, yaitu dengan membuat node baru dan mengarahkan pointer node tersebut ke head lama sehingga menjadi head yang baru.

```
package pekan5_2511531016;

public class TambahSLL_2511531016 {
public static NodeSLL_2511531016
insertAtFront_1016(NodeSLL_2511531016 head_1016, int value_1016) {
NodeSLL_2511531016 new_node = new NodeSLL_2511531016(value_1016);
```

```
new_node.next_1016 = head_1016;
return new_node;
```

Gambar 2.5 (Langkah 2 TambahSLL)

3. Selanjutnya, buat method untuk menambahkan node di akhir linked list, di mana program membuat node baru dan mengecek apakah linked list masih kosong.

```
// Fungsi menambahkan node di akhir SLL
public static NodeSLL_2511531016 insertAtEnd_1016(NodeSLL_2511531016
head_1016, int value_1016) {
// Buat sebuah node dengan sebuah nilai
NodeSLL_2511531016 newNode = new NodeSLL_2511531016(value_1016);
// Jika list kosong maka node jadi head
if (head_1016 == null) {
return newNode;
```

Gambar 2.6 (Langkah 3 TambahSLL)

4. Jika list tidak kosong, simpan head ke variabel sementara lalu menelusuri hingga node terakhir dan menghubungkannya dengan node baru.

```
// Simpan head ke variabel sementara
NodeSLL_2511531016 last = head_1016;
// Telusuri ke node akhir
while (last.next_1016 != null) {
last = last.next_1016;
}
// Ubah pointer
last.next_1016 = newNode;
return head_1016;
```

Gambar 2.7 (Langkah 4 TambahSLL)

5. Kemudian, buat method untuk membuat node baru secara langsung menggunakan fungsi GetNode_1016 yang mengembalikan node dengan nilai tertentu.

```
static NodeSLL_2511531016 GetNode_1016(int data_1016) {
```

```
return new NodeSLL_2511531016(data_1016);
}
```

Gambar 2.8 (Langkah 5 TambahSLL)

6. Buat method untuk menyisipkan node di posisi tertentu, dimulai dengan menyimpan head dan mengecek apakah posisi valid atau berada di awal.

```
static NodeSLL_2511531016 insertPos_1016(NodeSLL_2511531016 headNode,
int position_1016, int value_1016) {
NodeSLL_2511531016 head_1016 = headNode;
if (position_1016 < 1)
System.out.print ("Invalid position");
if (position_1016 == 1) {
NodeSLL_2511531016 new_node = new NodeSLL_2511531016(value_1016);
new_node.next_1016 = head_1016;
return new_node;
```

Gambar 2.9 (Langkah 6 TambahSLL)

7. Jika posisi bukan di awal, telusuri node satu per satu hingga posisi yang diinginkan lalu menyisipkan node baru di tengah linked list.

```
} else {
while (position_1016-- != 0) {
if (position_1016 == 1) {
NodeSLL_2511531016 newNode = GetNode_1016 (value_1016);
newNode.next_1016 = headNode.next_1016;
headNode.next_1016 = newNode;
break;
}
headNode = headNode.next_1016;
}
if (position_1016 != 1)
System.out.print("Posisi di luar jangkauan");
return head_1016;
```

Gambar 2.10 (Langkah 7 TambahSLL)

8. Buat method untuk mencetak isi linked list dengan menelusuri setiap node dari awal hingga akhir.

```
public static void printList_1016 (NodeSLL_2511531016 head_1016) {
NodeSLL_2511531016 curr_1016 = head_1016;
while (curr_1016.next_1016 != null) {
System.out.print(curr_1016.data_1016+"-->");
curr_1016 = curr_1016.next_1016;
}
if (curr_1016.next_1016==null) {
System.out.print(curr_1016.data_1016); }
System.out.println();
}
```

Gambar 2.11 (Langkah 8 TambahSLL)

9. Selanjutnya, pada method main, buat linked list awal dan cetak kondisi awal list.

```
public static void main (String[] args) {
// Buat linked list 2->3->5->6
NodeSLL_2511531016 head_1016 = new NodeSLL_2511531016(2);
head_1016.next_1016 = new NodeSLL_2511531016(3);
head_1016.next_1016.next_1016 = new NodeSLL_2511531016(5);
head_1016.next_1016.next_1016.next_1016 = new NodeSLL_2511531016(6);
// Cetak list asli
System.out.print("Senarai berantai awal:");
printList_1016 (head_1016);
```

Gambar 2.12 (Langkah 9 TambahSLL)

10. Tambahkan node di bagian depan menggunakan method insertAtFront_1016 lalu cetak hasilnya.

```
// Tambahkan node baru didepan
System.out.print("tambah 1 simpul di depan: ");
int data_1016 = 1;
head_1016 = insertAtFront_1016(head_1016, data_1016);
// Cetak update list
printList_1016(head_1016);
```

Gambar 2.13 (Langkah 10 TambahSLL)

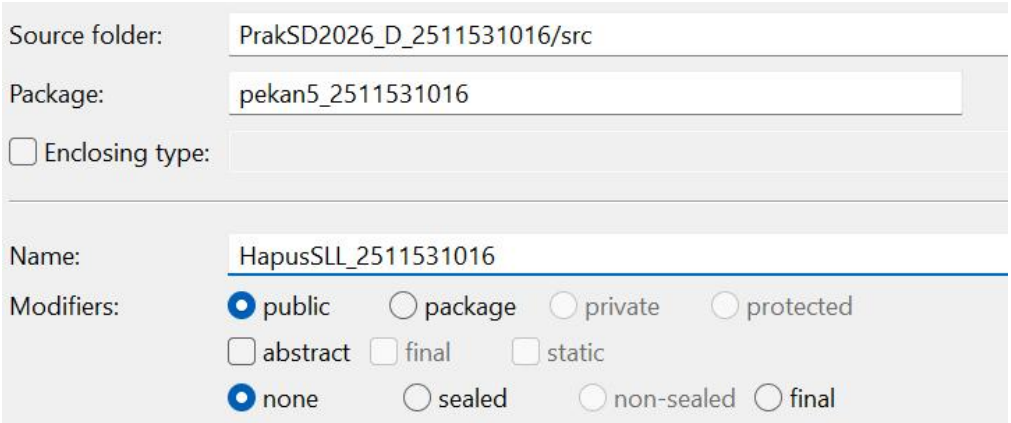
11. Terakhir, tambahkan node di bagian belakang dan pada posisi tertentu, kemudian cetak hasil akhir linked list.

```
// Tambahkan node baru dibelakang
System.out.print("tambah 1 simpul di belakang");
int data2 = 7;
head_1016 = insertAtEnd_1016 (head_1016, data2);
// Cetak update list
printList_1016 (head_1016);
System.out.print("tambah 1 simpul ke data 4: ");
int data3 = 4;
int pos = 4;
head_1016 = insertPos_1016 (head_1016, pos, data3);
// Cetak update list
printList_1016 (head_1016);
```

Gambar 2.14 (Langkah 11 TambahSLL)

2.3.3 HapusSLL_2511531016

1. Membuat kelas baru bernama *HapusSLL_2511531016* dalam *package pekan5_2511531016.*



Gambar 2.15 (Langkah 1 HapusSLL)

2. Program dimulai dengan deklarasi package dan class. Lalu, buat method untuk menghapus node di bagian depan (head), yaitu dengan mengecek apakah list kosong lalu memindahkan head ke node berikutnya sebagai head baru.

```
package pekan5_2511531016;

public class HapusSLL_2511531016 {
```

```
// Fungsi untuk menghapus head
public static NodeSLL_2511531016 deleteHead_1016 (NodeSLL_2511531016
head_1016) {
// Jika SLL kosong
if (head_1016 == null)
return null;
// Pindahkan head ke node berikutnya
head_1016 = head_1016.next_1016;
// Return head baru
return head_1016;
```

Gambar 2.16 (Langkah 2 HapusSLL)

3. Selanjutnya, buat method untuk menghapus node terakhir, dimulai dengan pengecekan apakah linked list kosong atau tidak.

```
// Fungsi menghapus node terakhir SLL
public static NodeSLL_2511531016 removeLastNode_1016
(NodeSLL_2511531016 head_1016) {
// Jika list kosong, return null
if (head_1016 == null) {
return null;
}
// Jika list satu node, hapus node dan return null
if (head_1016 == null) {
return null;
```

Gambar 2.17 (Langkah 3 HapusSLL)

4. Kemudian, program mencari node kedua terakhir dengan menelusuri list, lalu menghapus node terakhir dengan mengubah pointer menjadi null.

```
// Temukan node terakhir ke dua
NodeSLL_2511531016 secondLast = head_1016;
while (secondLast.next_1016.next_1016 != null) {
secondLast = secondLast.next_1016;
}
// Hapus node terakhir
secondLast.next_1016 = null;
return head_1016;
```

Gambar 2.18 (Langkah 4 HapusSLL)

5. Buat method untuk menghapus node pada posisi tertentu dengan menyiapkan variabel bantu dan mengecek apakah list kosong.

```
// Fungsi menghapus node di posisi tertentu
public static NodeSLL_2511531016 deleteNode_1016 (NodeSLL_2511531016
head_1016, int position_1016) {
NodeSLL_2511531016 temp_1016 = head_1016;
NodeSLL_2511531016 prev_1016 = null;
// Jika linked list null
if (temp_1016 == null)
return head_1016;
```

Gambar 2.19 (Langkah 5 HapusSLL)

6. Jika posisi = 1 maka node head dihapus dengan memindahkan head ke node berikutnya.

```
// Kasus 1: head dihapus
if (position_1016 == 1) {
head_1016 = temp_1016.next_1016;
return head_1016;
}
```

Gambar 2.20 (Langkah 6 HapusSLL)

7. Jika node yang dihapus berada di tengah, telusuri hingga posisi yang diinginkan menggunakan perulangan.

```
// Kasus 2: menghapus node di tengah
// Telusuri ke node yang dihapus
for (int i_1016 = 1; temp_1016 != null && i_1016 < position_1016;
i_1016++) {
prev_1016 = temp_1016;
temp_1016 = temp_1016.next_1016;
}
```

Gambar 2.21 (Langkah 7 HapusSLL)

8. Jika node ditemukan maka node dihapus dengan mengubah pointer, jika tidak maka tampilkan pesan bahwa data tidak ada.

```
// jika ditemukan, hapus node
if (temp_1016 != null) {
prev_1016.next_1016 = temp_1016.next_1016;
} else {
System.out.println("Data tidak ada");
}
return head_1016;
```

Gambar 2.22 (Langkah 8 HapusSLL)

9. Buat method untuk mencetak isi linked list dengan menelusuri setiap node dari awal hingga akhir.

```
// Fungsi mencetak SLL
public static void printList_1016(NodeSLL_2511531016 head_1016) {
NodeSLL_2511531016 curr_1016 = head_1016;
while (curr_1016.next_1016 != null) {
System.out.print(curr_1016.data_1016 + "-->");
curr_1016 = curr_1016.next_1016;
}
System.out.print(curr_1016.data_1016);
System.out.println();
```

Gambar 2.23 (Langkah 9 HapusSLL)

10. Selanjutnya, pada method main, buat linked list awal (1→2→3→4→5→6) dan cetak kondisi awal list.

```
// Kelas main
public static void main(String[] args) {
// Buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
NodeSLL_2511531016 head_1016 = new NodeSLL_2511531016(1);
head_1016.next_1016 = new NodeSLL_2511531016(2);
head_1016.next_1016.next_1016 = new NodeSLL_2511531016(3);
head_1016.next_1016.next_1016.next_1016 = new NodeSLL_2511531016(4);
head_1016.next_1016.next_1016.next_1016.next_1016 = new
NodeSLL_2511531016(5);
head_1016.next_1016.next_1016.next_1016.next_1016.next_1016 = new
NodeSLL_2511531016(6);

// Cetak list awal
System.out.println("List awal: ");
printList_1016(head_1016);
```

Gambar 2.24 (Langkah 10 HapusSLL)

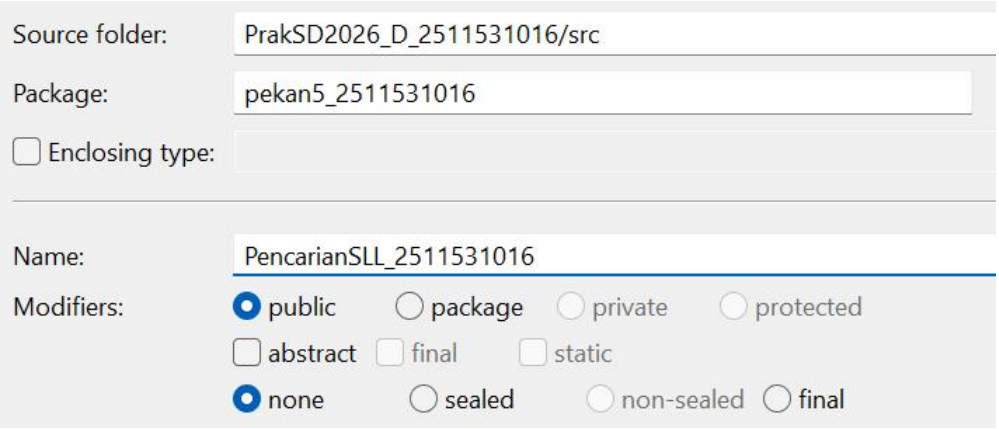
11. Terakhir, lakukan beberapa operasi penghapusan seperti menghapus head, node terakhir, dan node pada posisi tertentu, lalu cetak hasilnya setiap kali perubahan terjadi.

```
// Hapus head
head_1016 = deleteHead_1016(head_1016);
System.out.println("List setelah head dihapus: ");
printList_1016(head_1016);

// Hapus node terakhir
head_1016 = removeLastNode_1016(head_1016);
System.out.println("List setelah simpul terakhir dihapus: ");
printList_1016(head_1016);

// Hapus node di posisi 2
int position_1016 = 2;
head_1016 = deleteNode_1016(head_1016, position_1016);
System.out.println("List setelah posisi 2 dihapus: ");
printList_1016(head_1016);
```

Gambar 2.25 (Langkah 11 HapusSLL)

2.3.4 PencarianSLL_2511531016

1. Membuat kelas baru bernama *PencarianSLL_2511531016* dalam *package pekan5_2511531016.*



Gambar 2.26 (Langkah 1 PencarianSLL)

2. Program dimulai dengan deklarasi package dan class. Lalu, buat method searchKey_1016 untuk mencari data dalam linked list dengan cara menelusuri node satu per satu mulai dari head.

```
package pekan5_2511531016;

public class PencarianSLL_2511531016 {
static boolean searchKey_1016 (NodeSLL_2511531016 head_1016, int
key_1016) {
NodeSLL_2511531016 curr_1016 = head_1016;
while (curr_1016 != null) {
```

Gambar 2.27 (Langkah 2 PencarianSLL)

3. Selanjutnya, pada proses penelusuran lakukan pengecekan apakah data pada node sama dengan nilai yang dicari, jika sama maka langsung kembalikan nilai true, jika tidak maka lanjut ke node berikutnya hingga habis dan kembalikan false.

```
if (curr_1016.data_1016 == key_1016)
return true;
curr_1016 = curr_1016.next_1016;
}
return false;
```

Gambar 2.28 (Langkah 3 PencarianSLL)

4. Kemudian, buat method traversal_1016 untuk menampilkan isi linked list dengan cara menelusuri dari head hingga node terakhir sambil mencetak setiap data.

```
public static void traversal_1016 (NodeSLL_2511531016 head_1016) {
// Mulai dari head
NodeSLL_2511531016 curr_1016 = head_1016;
// Telusuri sampai pointer null
while (curr_1016 != null) {
System.out.print(" " + curr_1016.data_1016);
curr_1016 = curr_1016.next_1016;
}
System.out.println();
```

Gambar 2.29 (Langkah 4 PencarianSLL)

5. Setelah itu, pada method main, buat linked list awal dengan beberapa node lalu tampilkan isinya menggunakan fungsi traversal.

```
public static void main (String[] args) {
NodeSLL_2511531016 head_1016 = new NodeSLL_2511531016(14);
head_1016.next_1016 = new NodeSLL_2511531016(21);
head_1016.next_1016.next_1016 = new NodeSLL_2511531016(13);
head_1016.next_1016.next_1016.next_1016 = new NodeSLL_2511531016(30);
head_1016.next_1016.next_1016.next_1016.next_1016 = new
NodeSLL_2511531016(10);
System.out.print ("Penelusuran SLL : ") ;
traversal_1016 (head_1016);
```

Gambar 2.30 (Langkah 5 PencarianSLL)

6. Terakhir, program menentukan data yang akan dicari lalu memanggil method searchKey_1016, jika data ditemukan maka ditampilkan "ketemu", jika tidak maka "tidak ada".

```
// Data yang akan dicari
int key_1016 = 30;
System.out.print("cari data " + key_1016 + " = ");
if (searchKey_1016(head_1016, key_1016))
System.out.println("ketemu");
else
System.out.println("tidak ada");
```

Gambar 2.31 (Langkah 6 PencarianSLL)

2.4 Analisis Hasil

Berdasarkan hasil praktikum yang telah dilakukan, dapat dilihat bahwa keseluruhan program berhasil mengimplementasikan operasi dasar pada struktur *Single Linked List* secara lengkap. Proses dimulai dari pembuatan node melalui kelas *NodeSLL*, yang menjadi dasar pembentukan list dengan menyimpan data dan pointer ke node berikutnya. Selanjutnya, pada kelas *TambahSLL*, program mampu menambahkan node di berbagai posisi, baik di depan, di akhir, maupun di posisi tertentu, yang menunjukkan bahwa manipulasi pointer sudah dilakukan dengan benar. Selain itu, pada kelas *HapusSLL*, program juga dapat menghapus node di bagian awal, akhir, maupun posisi tertentu, sehingga memperlihatkan bagaimana struktur list dapat diperbarui secara dinamis tanpa merusak hubungan antar node.

Di sisi lain, kelas *PencarianSLL* melengkapi operasi dengan menyediakan fungsi *traversal* dan pencarian data. Traversal digunakan untuk menampilkan seluruh isi linked list secara berurutan, sedangkan pencarian dilakukan dengan metode linear search yang membandingkan setiap elemen hingga data ditemukan atau mencapai akhir list. Hasil yang diperoleh menunjukkan bahwa seluruh operasi, mulai dari penambahan, penghapusan, penelusuran, hingga pencarian, berjalan sesuai dengan konsep dasar *Single Linked List*. Secara keseluruhan, praktikum ini membantu memahami tidak hanya struktur data itu sendiri, tetapi juga bagaimana setiap operasi saling terhubung dalam mengelola data secara fleksibel dan terstruktur.

BAB III
KESIMPULAN

3.1 Hasil Praktikum

3.1.1 TambahSLL_2511531016

```
Problems   @ Javadoc   Declaration   Console ×
<terminated> TambahSLL_2511531016 [Java Application] C:\Users\User\.p2
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

Gambar 3.1 (Hasil Praktikum TambahSLL)

Output program tersebut menunjukkan perubahan isi linked list setelah dilakukan beberapa operasi penambahan node secara bertahap. Awalnya, list berisi urutan 2→3→5→6 yang ditampilkan sebagai kondisi awal. Ketika ditambahkan satu simpul di depan dengan nilai 1, posisi head berpindah sehingga urutannya menjadi 1→2→3→5→6.

Selanjutnya, penambahan di bagian belakang dengan nilai 7 membuat elemen baru muncul di akhir tanpa mengubah urutan sebelumnya, menjadi 1→2→3→5→6→7. Terakhir, saat ditambahkan nilai 4 pada posisi tertentu (di antara 3 dan 5), terlihat bahwa node baru berhasil disisipkan di tengah list sehingga urutannya menjadi 1→2→3→4→5→6→7. Dari sini bisa dipahami bahwa setiap operasi bekerja sesuai tujuan, baik di depan, belakang, maupun di posisi tertentu dalam linked list.

3.1.2 HapusSLL_2511531016

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> HapusSLL_2511531016 [Java Application] C:\Users
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 3.2 (Hasil Praktikum HapusSLL)

Output di atas terlihat bahwa perubahan isi linked list setelah dilakukan beberapa operasi penghapusan. Awalnya, list berisi urutan 1→2→3→4→5→6 sebagai kondisi awal. Setelah head dihapus, elemen pertama (1) hilang sehingga list menjadi 2→3→4→5→6. Kemudian, saat simpul terakhir dihapus, elemen paling akhir (6) juga ikut terhapus dan menyisakan 2→3→4→5. Terakhir, ketika node pada posisi ke-2 dihapus, elemen pada posisi tersebut (3) dihilangkan sehingga urutan berubah menjadi 2→4→5. Dari hasil ini terlihat bahwa setiap operasi penghapusan berhasil mengubah struktur linked list sesuai dengan posisi yang ditentukan.

3.1.3 PencarianSLL_2511531016

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> PencarianSLL_2511531016 [Java Application] C:\Us
Penelusuran SLL :  14 21 13 30 10
cari data 30  =  ketemu
```

Gambar 3.3 (Hasil Praktikum PencarianSLL)

Output program menunjukkan bahwa proses penelusuran (traversal) berhasil menampilkan seluruh isi linked list secara berurutan, yaitu 14, 21, 13, 30, dan 10, sesuai dengan data yang dimasukkan di awal. Setelah itu, program melakukan pencarian terhadap nilai 30 dengan membandingkannya pada setiap node. Karena nilai tersebut memang ada di dalam list, proses pencarian berhenti saat data ditemukan dan menghasilkan output “ketemu”. Hal ini menandakan bahwa mekanisme traversal dan pencarian berjalan sesuai dengan alur yang diharapkan.

3.2 Saran Pengembangan

Sebagai pengembangan dari praktikum yang telah dilakukan, program masih bisa dibuat lebih rapi dan aman dengan menambahkan validasi pada beberapa kondisi yang berpotensi menimbulkan error. Misalnya, pada proses penghapusan node terakhir, perlu ditambahkan pengecekan khusus jika linked list hanya memiliki satu node agar tidak terjadi kesalahan saat mengakses pointer berikutnya. Selain itu, pada method penyisipan dan penghapusan berdasarkan posisi, akan lebih baik jika penanganan posisi di luar jangkauan dibuat lebih konsisten, misalnya dengan mengembalikan nilai tertentu atau memberikan pesan yang lebih informatif, sehingga alur program lebih mudah dipahami saat terjadi kesalahan input.

Di sisi lain, struktur program juga bisa dikembangkan agar lebih modular dan fleksibel. Saat ini, setiap operasi masih ditulis dalam bentuk method statis, sehingga bisa dipertimbangkan untuk mengubahnya menjadi bagian dari satu kelas Linked List yang utuh dengan atribut head sebagai properti utama. Dengan begitu, pengelolaan data menjadi lebih terorganisir dan mendekati konsep implementasi objek yang sebenarnya. Selain itu, program juga bisa ditambahkan fitur lain seperti pencarian posisi data, menghitung jumlah node, atau bahkan pengurutan sederhana, sehingga fungsionalitas linked list menjadi lebih lengkap dan tidak hanya terbatas pada operasi dasar saja.

DAFTAR PUSTAKA

[1] R. Munir, *Struktur Data dan Algoritma dalam Pemrograman*, edisi revisi. Bandung: Informatika, 2022.

[2] A. Kadir, *Dasar Pemrograman Java dan Struktur Data*, Yogyakarta: Andi, 2021.

[3] B. Raharjo, *Belajar Pemrograman Java untuk Pemula dan Lanjut*, Bandung: Informatika, 2022.

[4] S. Wibowo dan D. Prasetyo, *Algoritma dan Struktur Data: Teori dan Implementasi*, Jakarta: Elex Media Komputindo, 2023.

TUGAS PRAKTIKUM PEKAN 5

Simulasi Antrian Rumah Sakit

1. Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

2. Kode Program

2.1 Pasien_2511531016

```java
package pekan5_2511531016;

public class Pasien_2511531016 {
    String namaPasien_1016;
    String penyakit_1016;
    int noAntrian_1016;
    Pasien_2511531016 next_1016;

    // konstruktor
    public Pasien_2511531016(String namaPasien_1016, String penyakit_1016, int noAntrian_1016) {
        this.namaPasien_1016 = namaPasien_1016;
        this.penyakit_1016 = penyakit_1016;
        this.noAntrian_1016 = noAntrian_1016;
        this.next_1016 = null;
    }

    // getter
    public String getNamaPasien_1016() {
        return namaPasien_1016;
    }
    public String getPenyakit_1016() {
        return penyakit_1016;
    }
    public int getNoAntrian_1016() {
        return noAntrian_1016;
    }
    public Pasien_2511531016 getNext_1016() {
        return next_1016;
    }

    // setter
    public void setNamaPasien_1016(String namaPasien_1016) {
        this.namaPasien_1016 = namaPasien_1016;
    }
    public void setPenyakit_1016(String penyakit_1016) {
```

```
        this.penyakit_1016 = penyakit_1016;
    }
    public void setNoAntrian_1016(int noAntrian_1016) {
        this.noAntrian_1016 = noAntrian_1016;
    }
    public void setNext_1016(Pasien_2511531016 next_1016) {
        this.next_1016 = next_1016;
    }
}
```

Penjelasan :

Kelas *Pasien_2511531016* merupakan kelas node yang digunakan sebagai struktur dasar dalam linked list pada sistem antrian rumah sakit. Setiap objek dari kelas ini merepresentasikan satu data pasien yang terdiri dari atribut *namaPasien_1016* untuk menyimpan nama pasien, *penyakit_1016* untuk menyimpan keluhan atau penyakit pasien, serta *noAntrian_1016* yang berfungsi sebagai nomor urut antrian. Selain itu, terdapat *pointer next_1016* yang digunakan untuk menghubungkan satu node pasien dengan *node* pasien berikutnya dalam struktur linked list.

Kelas ini juga memiliki konstruktor *Pasien_2511531016()* yang berfungsi untuk menginisialisasi data pasien saat objek dibuat, termasuk mengatur nilai *next_1016* menjadi null sebagai penanda bahwa *node* belum terhubung dengan *node* lain. Selain itu, terdapat method *getter* dan *setter* yang digunakan untuk mengakses dan mengubah nilai setiap atribut secara terkontrol. Dengan adanya *getter* dan *setter*, data pasien menjadi lebih aman karena tidak diakses langsung dari luar kelas, sehingga kelas ini berperan sebagai representasi satu elemen *(node)* yang membentuk keseluruhan struktur antrian pada linked list.

2.2 RumahSakit_2511531016

```java
package pekan5_2511531016;
import java.util.Scanner;

public class RumahSakit_2511531016 {
    Pasien_2511531016 head_1016 = null;
    int counter_1016 = 0;

    // insert at tail
    public void daftarkanPasien_1016(String nama_1016, String penyakit_1016) {
        counter_1016++;
        Pasien_2511531016 pasienBaru_1016 = new Pasien_2511531016(nama_1016, penyakit_1016, counter_1016);
        if (head_1016 == null) {
            head_1016 = pasienBaru_1016;
        } else {
            Pasien_2511531016 temp_1016 = head_1016;
            while (temp_1016.next_1016 != null) {
                temp_1016 = temp_1016.next_1016;
            }
            temp_1016.next_1016 = pasienBaru_1016;
        }
        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_1016);
    }

    // delete head
    public void panggilPasien_1016() {
        if (head_1016 == null) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println("Memanggil Pasien:");
        System.out.println("Nama: " + head_1016.namaPasien_1016);
        System.out.println("Keluhan: " + head_1016.penyakit_1016);
        System.out.println("No Antrian: " + head_1016.noAntrian_1016);
        head_1016 = head_1016.next_1016;
    }
```

```java
// display
public void tampilkanAntrian_1016() {
    if (head_1016 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
    Pasien_2511531016 temp_1016 = head_1016;
    System.out.println("Daftar Antrian:");
    while (temp_1016 != null) {
        System.out.println("No: " + temp_1016.noAntrian_1016);
        System.out.println("Nama: " + temp_1016.namaPasien_1016);
        System.out.println("Keluhan: " + temp_1016.penyakit_1016);
        temp_1016 = temp_1016.next_1016;
    }
}
// search
public void cariPasien_1016(String nama_1016) {
    Pasien_2511531016 temp_1016 = head_1016;
    while (temp_1016 != null) {
        if (temp_1016.namaPasien_1016.equalsIgnoreCase(nama_1016)) {
            System.out.println("Pasien ditemukan!");
            System.out.println("No: " + temp_1016.noAntrian_1016);
            System.out.println("Nama: " + temp_1016.namaPasien_1016);
            System.out.println("Keluhan: " + temp_1016.penyakit_1016);
            return;
        }
        temp_1016 = temp_1016.next_1016;
```

```java
    }
    System.out.println("Pasien tidak ditemukan!");
}

// status
public void cekStatus_1016() {
    if (head_1016 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
    int jumlah_1016 = 0;
    Pasien_2511531016 temp_1016 = head_1016;

    while (temp_1016 != null) {
        jumlah_1016++;
        temp_1016 = temp_1016.next_1016;
    }
    System.out.println("Jumlah pasien: " + jumlah_1016);
    System.out.println("Pasien terdepan: " + head_1016.namaPasien_1016);
    System.out.println("No: " + head_1016.noAntrian_1016);
}

public static void main(String[] args) {
    Scanner input_1016 = new Scanner(System.in);
    RumahSakit_2511531016 rs_1016 = new RumahSakit_2511531016();
    int choice_1016;

    do {
        System.out.println("\n=== Antrian Rumah Sakit NIM: 2511531016 ===");
        System.out.println("1. Daftarkan Pasien");
        System.out.println("2. Panggil Pasien");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Cari Pasien");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");

        System.out.print("Pilihan: ");
        choice_1016 = input_1016.nextInt();
        input_1016.nextLine();

        switch (choice_1016) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama_1016 = input_1016.nextLine();
                System.out.print("Masukkan Keluhan: ");
                String keluhan_1016 = input_1016.nextLine();
                rs_1016.daftarkanPasien_1016(nama_1016, keluhan_1016);
                break;
            case 2:
                rs_1016.panggilPasien_1016();
                break;
            case 3:
                rs_1016.tampilkanAntrian_1016();
                break;
            case 4:
                System.out.print("Masukkan nama yang dicari: ");
                String cari_1016 = input_1016.nextLine();
                rs_1016.cariPasien_1016(cari_1016);
                break;
            case 5:
                rs_1016.cekStatus_1016();
                break;
            case 6:
                System.out.println("Terima kasih!");
                break;

            default:
                System.out.println("Pilihan tidak valid!");
        }
```

```
        } while (choice_1016 != 6);
        input_1016.close();
    }
}
```

Penjelasan :

Kelas *RumahSakit_2511531016* merupakan kelas utama yang berfungsi sebagai pengelola sistem antrian pasien menggunakan struktur data linked list. Di dalam kelas ini terdapat variabel utama *head_1016* yang berperan sebagai pointer awal *(head)* dari linked list, serta *counter_1016* yang digunakan untuk memberikan nomor antrian secara otomatis setiap kali pasien baru didaftarkan. Kelas ini juga menjadi pusat logika program karena seluruh proses pengelolaan data pasien dilakukan melalui method-method yang ada di dalamnya.

Selain variabel, kelas ini memiliki beberapa method utama seperti *daftarkanPasien_1016()* untuk menambahkan data pasien ke antrian, *panggilPasien_1016()* untuk menghapus dan memanggil pasien terdepan, *tampilkanAntrian_1016()* untuk menelusuri seluruh data dalam linked list, *cariPasien_1016()* untuk melakukan pencarian berdasarkan nama pasien, serta *cekStatus_1016()* untuk menampilkan jumlah pasien dan kondisi antrian saat ini. Pada bagian akhir terdapat method *main()* yang berfungsi sebagai antarmuka program, yaitu menampilkan menu interaktif dan menghubungkan input pengguna dengan operasi pada *linked list*.

3. Output Kode Program

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> RumahSakit_2511531016 [Java Application] C:\Users\
=== Antrian Rumah Sakit NIM: 2511531016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Cata
Masukkan Keluhan: Flu
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511531016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Lina
Masukkan Keluhan: Nyeri punggung
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511531016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil Pasien:
Nama: Cata
Keluhan: Flu

No Antrian: 1

=== Antrian Rumah Sakit NIM: 2511531016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Daftar Antrian:
No: 2
Nama: Lina
Keluhan: Nyeri punggung

=== Antrian Rumah Sakit NIM: 2511531016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama yang dicari: Lina
Pasien ditemukan!
No: 2
Nama: Lina
Keluhan: Nyeri punggung
```

```
=== Antrian Rumah Sakit NIM: 2511531016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien: 1
Pasien terdepan: Lina
No: 2

=== Antrian Rumah Sakit NIM: 2511531016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Terima kasih!
```

Penjelasan :

Output program menunjukkan proses pengelolaan antrian pasien menggunakan struktur data linked list pada sistem Rumah Sakit. Pada awalnya, pengguna melakukan pendaftaran dua pasien, yaitu Cata dengan keluhan flu dan Lina dengan keluhan nyeri punggung. Setiap pasien yang didaftarkan akan ditambahkan ke dalam antrian secara berurutan, di mana Cata menjadi pasien pertama dengan nomor antrian 1, kemudian disusul Lina dengan nomor antrian 2. Hal ini menunjukkan bahwa proses insert pada linked list berjalan secara FIFO *(First In First Out)* pada bagian akhir *(tail)*.

Selanjutnya, pasien Cata dipanggil terlebih dahulu menggunakan menu "Panggil Pasien", sehingga node pertama dihapus dari antrian dan posisi head berpindah ke Lina. Ketika antrian ditampilkan kembali, hanya tersisa satu pasien yaitu Lina. Program juga berhasil melakukan pencarian data pasien Lina serta menampilkan status antrian yang tersisa sebanyak 1 orang dengan Lina sebagai pasien terdepan. Terakhir, sistem ditutup melalui menu keluar, yang menandakan seluruh operasi pada linked list telah berjalan dengan baik mulai dari insert, delete, search, hingga traversal.